

 Part 1 – Question Paper**10-Mark Questions**

1. Explain the complete Hadoop Architecture, detailing the roles of master and slave nodes.
2. Describe the HDFS Architecture and how it manages distributed storage and data replication.
3. Elaborate on the YARN Architecture, highlighting its core components and their functions.
4. Discuss the MapReduce Programming Model, explaining the workflow from input to final output.
5. Analyze Hadoop Implementation Challenges and the future trends of the ecosystem.

5-Mark Questions

6. Define Hadoop and its primary Ecosystem Components such as Hive and YARN.
7. Explain the roles of NameNode and DataNode in the context of file system operations.
8. Describe the Resource Manager in YARN and its two major internal components.
9. Compare FIFO, Capacity, and Fair Schedulers used in the YARN framework.
10. Explain the workflow of an Application Master from launch to shutdown.
11. Describe the Map Phase of the MapReduce algorithm in detail.
12. Discuss the use cases of Hadoop across different industrial sectors like Healthcare and Retail.
13. Explain the concept of Containers and their importance in resource management.
14. Describe the cost-effectiveness of Hadoop, comparing open-source and commercial versions.
15. How does HDFS ensure Reliability and Scalability within a cluster?

2-Mark Questions

16. What is the Master-Slave architecture in Hadoop?
17. Define Metadata as it relates to HDFS.
18. What is the role of Hive in the Hadoop ecosystem?
19. Briefly explain the Sort and Shuffle phase of MapReduce.
20. What is YARN and why was it introduced?
21. Define a DataNode.
22. What is the primary function of the Node Manager?
23. Mention two industries where Hadoop is applied.
24. Identify the two major components of the Resource Manager.
25. What does replication mean in HDFS?

EXAM POV NOTES (Q&A FORMAT)**✦ Part 2 – Complete Answer Bank****10-Mark Questions****1. Explain the complete Hadoop Architecture, detailing the roles of master and slave nodes.**

Hadoop is a robust open-source framework designed for the distributed storage and parallel processing of massive datasets across clusters of computers. It utilizes a structured architecture to handle both unstructured and semi-structured data effectively for business insights.

- **Master-Slave Framework:** Hadoop operates on a master-slave architecture where central master nodes control the distribution of work to multiple slave nodes. This design allows the system to scale effectively by adding more hardware as data volumes grow.
- **Role of Master Nodes:** The master nodes, such as the NameNode and ResourceManager, are responsible for managing metadata and coordinating cluster resources. They act as the "brain" of the cluster, deciding where data resides and which tasks are assigned.
- **Role of Slave Nodes:** Slave nodes, including DataNodes and NodeManagers, perform the actual heavy lifting by storing raw data and executing computational tasks. They follow instructions from the master nodes to read, write, or process data blocks.
- **HDFS Management:** The NameNode serves as the master of the Hadoop Distributed File System, regulating client access and managing the file system namespace. It maintains a record of every change made to the file system or its internal properties.
- **Resource Coordination:** The ResourceManager acts as the master daemon for YARN, overseeing the allocation of CPU and memory across all running applications. It ensures that the cluster is used optimally without any single application monopolising the hardware.
- **Data Storage (DataNodes):** In the slave layer, DataNodes manage the physical storage attached to the nodes they run on, handling block creation and deletion. They are responsible for serving all read and write requests directly from the system's clients.
- **Task Execution (NodeManagers):** Each slave node features a NodeManager that monitors local resource usage like RAM and CPU cores while reporting health status to the master. It manages the lifecycle of containers where individual tasks are executed.
- **Job Processing (MapReduce):** The architecture supports the MapReduce programming model, which simplifies the development of distributed applications through parallel processing. This model splits tasks into Map and Reduce phases to handle data across the cluster.
- **Fault Tolerance:** The architecture ensures reliability by replicating data blocks across multiple slave nodes to prevent loss during hardware failures. If a node fails, the master nodes automatically redirect tasks or recover data from replicas.
- **Scalability Features:** Hadoop's architecture is designed to scale horizontally, meaning organizations can increase capacity by simply adding more commodity servers. This allows for the management of thousands of nodes within a single integrated cluster.

2. Describe the HDFS Architecture and how it manages distributed storage and data replication.

The Hadoop Distributed File System (HDFS) is a specialised file system that provides highly reliable storage for large-scale datasets across a cluster. It is designed to be fault-tolerant and to run on low-cost, commodity hardware while maintaining high throughput.

- **Single NameNode Master:** Every HDFS cluster contains a single NameNode that manages the file system namespace and regulates client access to files. It is the central repository for all metadata, including file names and the mapping of blocks to nodes.
- **Multiple DataNode Slaves:** The architecture includes numerous DataNodes that manage the physical storage on individual machines within the cluster. These nodes perform the actual storage operations such as creating, deleting, and replicating data blocks.
- **Block-Based Storage:** HDFS stores files by splitting them into one or more large chunks called blocks, which are then distributed across several DataNodes. This allows very large files to be stored even if they exceed the capacity of a single disk.
- **Data Replication Strategy:** To ensure high availability, data is replicated across multiple nodes so that copies exist in different locations. A user can specify the number of replicas required to meet their specific data safety standards.
- **Namespace Operations:** The NameNode is responsible for executing namespace operations such as opening, closing, and renaming files or directories. It keeps a continuous record of these changes to maintain a consistent view of the file system.
- **Serving Client Requests:** DataNodes are responsible for serving the actual read and write requests from clients, ensuring data flows directly to the processing tasks. This prevents the NameNode from becoming a bottleneck for high-volume data traffic.
- **Fault Tolerance Design:** HDFS is built to handle node failures without data loss by automatically using redundant copies of blocks stored on other machines. The system constantly monitors node health to trigger recovery processes if a failure occurs.
- **Horizontal Scalability:** The system is designed to scale horizontally, allowing administrators to add more nodes as data requirements grow over time. This makes HDFS suitable for storing petabytes of data across thousands of machines.
- **Rack Awareness:** In a typical cluster, DataNodes are organized into racks, and HDFS uses this information to optimize data placement for better reliability. This ensures that at least one copy of data is stored on a different rack in case of a rack-level failure.
- **Reliability Mechanisms:** HDFS maintains reliability by frequently checking the integrity of stored blocks and ensuring the specified number of replicas always exists. The NameNode instructs DataNodes to perform replication if the count falls below the desired level.

3. Elaborate on the YARN Architecture, highlighting its core components and their functions.

YARN (Yet Another Resource Negotiator) is a resource management framework introduced in Hadoop 2.0 to separate resource management from job execution. It serves as the operating system for Hadoop, allowing multiple processing engines to run on the same cluster.

- **ResourceManager (RM):** The RM is the master daemon that holds global authority over managing and allocating cluster resources to all applications. It tracks the total availability of CPU and memory to ensure efficient distribution across the cluster.
- **NodeManager (NM):** Each worker node has an NM agent that manages local resources and monitors the health of the individual machine. It reports resource usage back to the RM and oversees the lifecycle of execution containers.
- **ApplicationMaster (AM):** Every submitted application has its own AM, which manages the specific execution life cycle of that job. It is responsible for negotiating resources from the RM and tracking task progress.
- **Container Concept:** A container is a collection of physical resources, such as RAM and CPU cores, on a single node where tasks are executed. They are supervised by the NodeManager and allocated dynamically by the ResourceManager.
- **The Scheduler Component:** Part of the RM, the Scheduler allocates resources based on pre-defined constraints like queue priority and hardware availability. It does not track the status of individual tasks or handle application-level failures.
- **Application Manager Component:** This RM component accepts job submissions from clients and launches the first container for the ApplicationMaster. It also provides a service to restart the AM if it fails during execution.
- **Resource Request Workflow:** The AM requests containers from the RM's Scheduler, which then assigns available slots on various NodeManagers. This dynamic negotiation allows for high cluster utilization even with varying workloads.
- **Task Monitoring:** The AM monitors the tasks running inside containers across different nodes and handles any local failures by retrying tasks. Once all tasks are complete, the AM notifies the RM and shuts itself down.
- **Fault Tolerance:** YARN provides reliability by allowing the RM to restart failed ApplicationMasters and the AM to request new containers for failed tasks. A standby RM can also take over if the primary RM fails.
- **Multi-framework Support:** YARN's decoupled architecture allows it to support various frameworks like Spark, Tez, and Flink alongside traditional MapReduce. This flexibility enables organizations to run diverse data processing workloads on a single cluster.

4. Discuss the MapReduce Programming Model, explaining the workflow from input to final output.

MapReduce is a programming model and processing technique designed for distributed computing, primarily based on the Java language. It enables the parallel processing of vast datasets by breaking jobs into manageable pieces across many nodes.

- **Distributed Model:** The MapReduce model simplifies the development of distributed applications by handling the complexities of parallelization and data movement. It allows developers to focus on the processing logic rather than the underlying cluster mechanics.
- **Input Data Splitting:** Before processing begins, the input data—typically stored in HDFS—is split into smaller chunks or blocks. This splitting allows multiple nodes to work on different parts of the same file simultaneously.
- **Map Phase Functions:** In the Map phase, "mapper" functions process these data chunks line by line to transform raw data into intermediate key-value pairs. This phase is responsible for initial filtering and transformation of the data.
- **Mapper Execution:** Mappers run on the slave nodes where the data blocks are physically stored to minimize network traffic. Each mapper produces a set of small data chunks that represent partial processing results.
- **Sort and Shuffle Phase:** Once mappers complete their tasks, the intermediate results are sorted by key to group identical items together. This phase acts as a bridge, collecting all values associated with a unique key for the next step.
- **Grouping Keys:** The output from the shuffle phase is formatted as <key, list(values)>, which ensures that the reducer receives all related data at once. This sorting is critical for the efficiency of the final aggregation.
- **Reduce Phase Functions:** "Reducer" functions take the grouped key-value pairs and perform computations like aggregations or summarizations. The reducer's job is to consolidate the intermediate data into a final result set.
- **Output Generation:** After processing is complete, the reducer produces a new set of output data that represents the final answer to the query. This output is typically stored back into the HDFS for further use or analysis.
- **Parallel Processing:** Both Map and Reduce phases happen in parallel across multiple computing nodes in the cluster. This parallelization is what allows Hadoop to process petabytes of data in a reasonable timeframe.
- **Scalability Benefits:** The major advantage of this model is its ease of scaling, as data processing can be distributed over thousands of nodes. This ensures that as data grows, the processing time remains manageable by adding more hardware.

5. Analyze Hadoop Implementation Challenges and the future trends of the ecosystem.

Implementing Hadoop in an enterprise environment involves significant complexities ranging from cluster management to ensuring data security. Despite these challenges, the ecosystem continues to evolve with trends like cloud integration and real-time processing.

- **Cluster Management Complexity:** Managing a large-scale Hadoop cluster is a significant challenge that requires specialized expertise in setup, configuration, and continuous monitoring. Organizations must invest in skilled personnel to maintain system health and performance.
- **Data Integration Hurdles:** Integrating data from diverse sources such as databases, logs, and social media can be complex due to varying formats. This requires intensive data cleaning, transformation, and the standardization of schemas before analysis.
- **Performance Optimization:** Tuning a cluster for large-scale processing is difficult and requires advanced techniques like data partitioning, indexing, and caching. Without proper optimization, processing times can become prohibitively slow for large datasets.
- **Security and Access Control:** Securing massive amounts of data in Hadoop necessitates strong authentication and authorization mechanisms. Preventing unauthorized access while allowing legitimate users to query data is a constant balancing act.
- **Cost Considerations:** While open-source Hadoop is free, the total cost of ownership including hardware, personnel, and maintenance can be high. Commercial distributions add further licensing fees but offer better support and management tools.
- **Trend - Cloud Integration:** There is an increasing shift toward adopting Hadoop on cloud platforms to gain scalability and cost-efficiency. Cloud services simplify deployment and reduce the need for maintaining physical on-premise hardware.
- **Trend - Real-Time Processing:** The ecosystem is moving beyond batch processing toward real-time analysis using frameworks like Spark Streaming and Kafka. This allows businesses to gain actionable intelligence from data as it arrives.
- **Trend - Machine Learning:** Hadoop is being increasingly integrated with machine learning algorithms for predictive analytics and fraud detection. This synergy allows companies to run complex models directly on their massive data stores.
- **Trend - Data Governance:** There is a growing emphasis on data governance and compliance to ensure privacy and regulatory standards are met. This involves tracking data lineage and maintaining strict control over sensitive information.
- **Future Outlook:** The future of Hadoop lies in its ability to adapt to hybrid environments and serve as a foundation for advanced AI applications. Its compatibility with modern distributed frameworks ensures it remains a core component of big data strategy.

5-Mark Questions

6. Define Hadoop and its primary Ecosystem Components such as Hive and YARN. Hadoop is an open-source framework that enables the distributed storage and processing of massive datasets across clusters of computers. It is specifically designed to handle unstructured and semi-structured data for valuable business insights.

- **HDFS (Storage):** The Hadoop Distributed File System provides a reliable, block-based storage layer that replicates data across nodes for fault tolerance.
- **YARN (Management):** Yet Another Resource Negotiator manages cluster resources and coordinates how applications execute across the available hardware.
- **MapReduce (Processing):** This programming model enables parallel data processing across multiple nodes, simplifying the development of distributed tasks.
- **Hive (Analysis):** Hive is a data warehouse system that provides a SQL-like interface, allowing users to query data stored in HDFS easily.
- **Spark/Pig (Advanced Tools):** The ecosystem also includes tools like Pig and Spark for more complex data transformations, real-time processing, and advanced analytics.

7. Explain the roles of NameNode and DataNode in the context of file system operations. HDFS uses a master-slave architecture to separate the management of the file system from the actual storage of data. This ensures that the system can handle large files and many concurrent clients efficiently.

- **Namespace Management (NameNode):** The NameNode is the master node that manages the file system namespace and records every change made to directories or files.
- **Metadata Storage (NameNode):** It stores all metadata, including file names, the number of replicas, and the mapping of file blocks to specific DataNodes.
- **Physical Storage (DataNode):** DataNodes are the slave nodes that manage the physical disks and store the actual data blocks assigned to them.
- **Serving Requests (DataNode):** They directly handle the read and write operations requested by clients, which keeps the NameNode free for management tasks.
- **Block Maintenance (Both):** The NameNode instructs DataNodes on block creation, deletion, and replication, while DataNodes execute these commands and report back.

8. Describe the Resource Manager in YARN and its two major internal components.

The ResourceManager (RM) is the master daemon in YARN responsible for managing and allocating resources across the entire Hadoop cluster. It oversees all applications and ensures resources are distributed according to policy.

- **Global Resource Management:** The RM tracks the availability of CPU and memory on every node to decide where new tasks should be launched.
- **Scheduler Component:** One major component is the Scheduler, which allocates resources based on constraints like memory limits and queue priorities.
- **Policy Enforcement (Scheduler):** The Scheduler uses various policies (FIFO, Capacity, or Fair) but does not monitor the actual execution of the tasks.
- **Application Manager Component:** The second major component is the Application Manager, which accepts job submissions from clients and starts the first container for the AM.
- **Failure Handling (Application Manager):** It is also responsible for monitoring the ApplicationMaster and restarting it on a new node if it fails during operation.

9. Compare FIFO, Capacity, and Fair Schedulers used in the YARN framework.

The YARN ResourceManager uses Schedulers to allocate resources based on different organizational needs and priorities. These policies determine how multiple jobs share the cluster's hardware resources.

Feature	FIFO Scheduler	Capacity Scheduler	Fair Scheduler
Ordering	Simple First In, First Out basis.	Multiple pre-defined job queues.	Dynamic priority-based sharing.
Sharing	Jobs wait in a single queue.	Occupants share a large cluster.	Resources shared dynamically.
Resources	One job uses all slots until done.	Dedicated slots per queue.	Distributed equally over time.
Priority	No inherent priority handling.	Queues can borrow free slots.	High priority jobs run in parallel.
Complexity	Default and simplest to use.	Requires pre-defined capacity.	Maintains resources dynamically.

10. Explain the workflow of an Application Master from launch to shutdown.

Every YARN application has its own ApplicationMaster (AM) that manages its execution from start to finish. It acts as a dedicated manager for a single job's lifecycle within the cluster.

- **Launch Phase:** The AM is launched in a container by the ResourceManager's Application Manager after a client submits a job.
- **Resource Negotiation:** Once active, the AM requests specific containers from the ResourceManager's Scheduler to run its required tasks.
- **Task Execution:** It manages these tasks on various NodeManagers, ensuring they have the resources needed to process their portion of the data.
- **Progress Monitoring:** The AM tracks task execution, handles failures by retrying tasks, and reports overall progress back to the client.
- **Shutdown Phase:** After all tasks are complete, the AM notifies the ResourceManager, shuts itself down, and releases its resources back to the cluster.

11. Describe the Map Phase of the MapReduce algorithm in detail.

The Map Phase is the first stage of data processing in the MapReduce model, responsible for transforming raw input into a structured format. It prepares the data for final aggregation in the Reduce stage.

- **Input Splitting:** Raw data from HDFS is split into smaller chunks so that multiple mappers can work on them in parallel across the cluster.
- **Mapper Function:** A user-defined mapper function processes each line of the input to extract relevant information.
- **Key-Value Pairs:** The primary output of the mapper is a set of intermediate key-value pairs representing transformed data points.
- **Local Processing:** Mappers typically run on the nodes where the data blocks are physically stored to avoid unnecessary network data movement.
- **Intermediate Chunks:** Each mapper creates several small chunks of data that are subsequently sorted and shuffled before the final reduction begins.

12. Discuss the use cases of Hadoop across different industrial sectors like Healthcare and Retail.

Hadoop allows businesses to ingest and analyze vast amounts of data from various sources to gain actionable intelligence. Its versatility makes it a valuable tool across many different sectors.

- **Healthcare Industry:** Hospitals use Hadoop to analyze medical records and identify disease patterns to improve overall patient care and outcomes.
- **Finance Sector:** Banks apply Hadoop for fraud detection, risk assessment, and detailed customer analytics to secure transactions.
- **Retail Business:** Retailers use it for customer segmentation, optimizing inventory management, and providing personalized product recommendations.
- **E-Commerce Platforms:** Online stores analyze user behavior and search history to optimize their search engines and shopping experiences.
- **General Ingestion:** Companies in these sectors ingest data from databases, logs, social media, and sensors directly into Hadoop for unified analysis.

13. Explain the concept of Containers and their importance in resource management. In the YARN architecture, a container is the basic unit of resource allocation where individual tasks are executed. They provide a way to isolate resources on a single node for a specific application.

- **Resource Collection:** A container represents a specific amount of physical resources, such as RAM, CPU cores, and disk space, on a node.
- **NodeManager Supervision:** Every container is strictly supervised by the local NodeManager to ensure it doesn't exceed its allocated resource limits.
- **Dynamic Sizing:** The ApplicationMaster can request containers of various sizes, typically in multiples of a minimum base size (e.g., 1 GB).
- **Task Environment:** Tasks are launched inside these containers, providing them with a dedicated environment to perform computations.
- **Efficient Utilization:** By using containers, YARN can pack multiple tasks from different applications onto the same physical server for better cluster utilization.

14. Describe the cost-effectiveness of Hadoop, comparing open-source and commercial versions.

The cost of implementing Hadoop can vary significantly depending on whether an organization chooses open-source or commercial distributions. Both options have different financial considerations for the business.

- **Open-Source Costs:** Apache Hadoop is free to download and use, making it an attractive "cheap" option for organizations with in-house expertise.
- **Infrastructure Investment:** Even with free software, businesses must invest heavily in hardware, infrastructure, and skilled personnel to manage the cluster.
- **Commercial Distributions:** Products like Cloudera or MapR come with licensing fees but provide enterprise support and easier management tools.
- **Reduced Expertise Need:** While more expensive, commercial versions can reduce the need for highly specialized in-house staff by providing expert support.
- **Cloud Scalability:** Using Hadoop in the cloud can be cost-effective as it allows organizations to pay only for the resources they use during processing.

15. How does HDFS ensure Reliability and Scalability within a cluster?

HDFS is specifically designed to reliably store very large files across many machines while allowing for easy growth. It uses several key strategies to maintain system integrity and performance.

- **Data Replication:** Every file is stored as a sequence of blocks that are replicated across multiple nodes for fault tolerance.
- **Horizontal Scaling:** HDFS scales by adding more commodity nodes to the cluster, which increases both storage capacity and processing throughput.
- **Automatic Recovery:** If a DataNode fails, the NameNode automatically detects the loss and instructs other nodes to re-replicate the missing blocks.
- **Distributed Storage:** By breaking files into chunks and storing them across many machines, HDFS avoids single points of failure for data access.
- **Namespace Records:** The NameNode maintains a persistent record of all changes, ensuring the file system remains consistent even after hardware reboots.

2-Mark Questions

16. What is the Master-Slave architecture in Hadoop?

- **Centralized Management:** The **master nodes** (NameNode, ResourceManager) are responsible for managing metadata, resources, and assigning tasks across the cluster.
- **Distributed Execution:** The **slave nodes** (DataNodes, NodeManagers) store the actual data blocks and perform the physical computation as instructed by the masters.

17. Define Metadata as it relates to HDFS.

- **File System Info: Metadata in HDFS** includes essential information such as file names, directory hierarchies, and the number of replicas required for each file.
- **Block Mapping:** It also contains the **mapping of blocks** to specific DataNodes, allowing the NameNode to know exactly where every part of a file is stored.

18. What is the role of Hive in the Hadoop ecosystem?

- **SQL-Like Interface:** Hive provides a familiar SQL-like interface that allows users to query and analyze massive datasets without writing complex Java code.
- **Data Warehousing:** It acts as a **data warehouse system** built on top of HDFS, simplifying the process of extracting insights from distributed data.

19. Briefly explain the Sort and Shuffle phase of MapReduce.

- **Key-Based Sorting:** This phase **collects and sorts** all the intermediate key-value pairs produced by mappers so that identical keys are grouped together.
- **Reducer Input:** It provides the **formatted input** <key, list(values)> to the reducers, ensuring all related data is processed by the same function.

20. What is YARN and why was it introduced?

- **Resource Framework:** YARN is a resource management framework introduced in Hadoop 2.0 to separate resource allocation from job monitoring.
- **Overcoming Limits:** It was **developed to overcome** the scalability limitations of earlier Hadoop versions and to support multiple processing engines on one cluster.

21. Define a DataNode.

- **Storage Manager:** A **DataNode manages** the actual storage hardware attached to the worker nodes and stores data as blocks.
- **Operation Execution:** It is responsible for **performing block creation**, deletion, and replication based on direct instructions from the master NameNode.

22. What is the primary function of the Node Manager?

- **Resource Monitoring:** The **Node Manager monitors** local resource usage such as CPU, memory, and disk health on an individual node.
- **Container Management:** It acts as a **per-node agent** that launches and oversees the containers where application tasks are executed.

23. Mention two industries where Hadoop is applied.

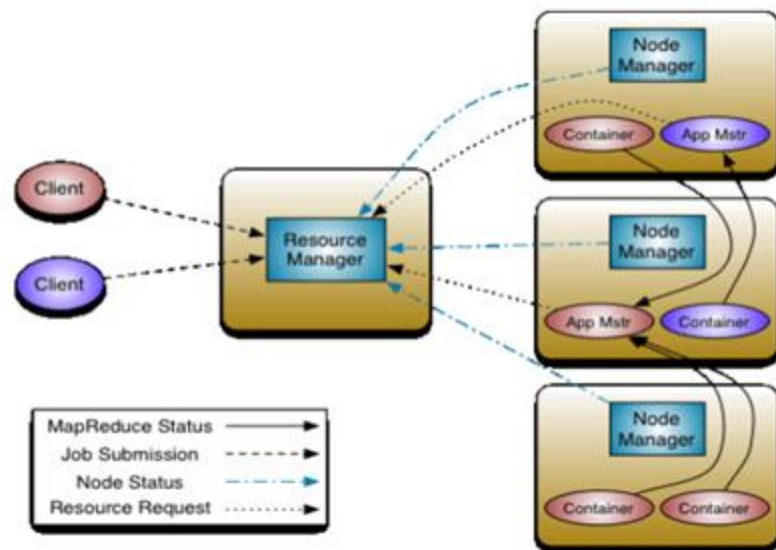
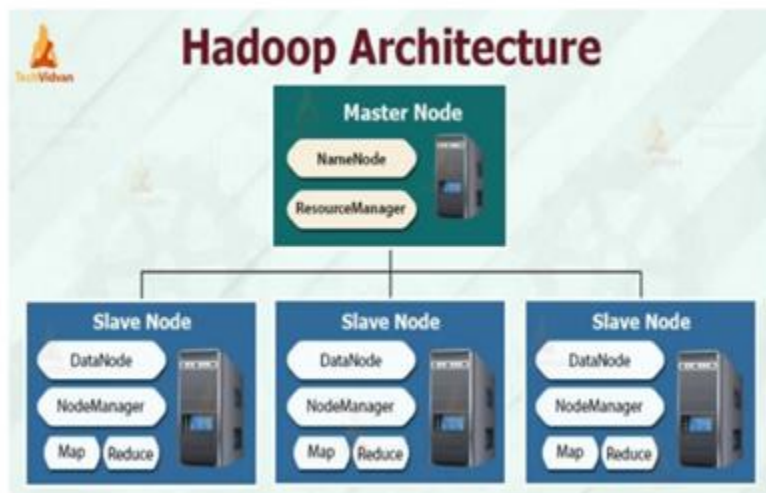
- **Healthcare:** This industry uses Hadoop to **analyze medical records** and identify disease patterns for better patient care.
- **Finance:** Banks and financial institutions apply Hadoop for **fraud detection** and conducting detailed risk assessments on large transaction volumes.

24. Identify the two major components of the Resource Manager.

- **Scheduler:** This component **allocates cluster resources** (CPU, RAM) to applications based on pre-defined policies and priorities.
- **Application Manager:** This component **accepts job submissions** and launches the initial container for an application's specific manager.

25. What does replication mean in HDFS?

- **Redundant Copies:** **Replication involves** creating multiple copies of the same data block and storing them across different machines in the cluster.
- **Fault Tolerance:** This process ensures **high data availability** and reliability, allowing the system to recover data even if multiple hardware nodes fail.



HDFS Architecture

